

Assignment 2 - Lectures 3 and 4

Instructions:

- The assignments are individual. You may consult and discuss with your colleagues, but you need to provide independent answers.
- You may use Matlab/Python/Julia to solve the assignment. Many questions require you clearly to do so (all practical problems in this set).
- Provide detailed answers, describing the steps you followed.
- Provide clear reports, typed, preferably using LaTeX.
- Submit the reports digitally on Brightspace as indicated on the lectures.
- Include your code, for reproducibility of your results, in your Brightspace submission, and possibly also through a link on your report.
- The code will *only* be used to verify reproducibility in case of doubts. The grading will be performed based on the results described in the report.

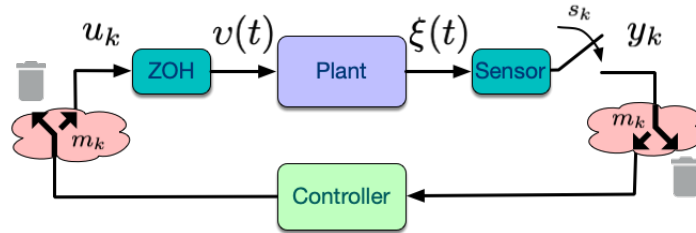


Figure 1: Schematic of a NCS including delays and packet-losses.

Consider a sample-data networked control system (NCS), as depicted in Figure 1. Assume that the plant dynamics are linear time-invariant given by:

$$\dot{\xi}(t) = A\xi(t) + Bv(t),$$

where the control signal v is a piece-wise constant signal resulting from the application of a controller in sampled-and-hold fashion, i.e. $v(t) = u_k, t \in [s_k, s_{k+1})$.

The system matrices are given by:

$$A = \begin{bmatrix} 0 & 0.5 - c \\ 0.2 + a - b & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

where a is given by the first digit, b by the third digit, and c by the last digit of your student ID number. **This is the same plant from Assignment 1.**

We revisit the problem in Questions 4 of Assignment 1. We consider as in Assignment 1 that the system is implemented with a varying sampling interval taking two possible values $h_1 = h$ and $h_2 = 2h$ but the choice of controller to be employed is not fixed. When the sampling interval is h_1 the controllers $\bar{K}$ or $\bar{K}_2$ are applied indistinctly; and when the interval is h_2 the control $\bar{K}_2$ or the zero input are applied indistinctly. The sequence of sampling intervals is fixed to be $(h_1 h_2)^\omega$.

Question 1: (10p)

1. (3p) Create an ω -automaton that models this behavior.
2. (3p) Write down the equations of the switched system describing the dynamics of the received samples using this network model.
3. (2p) Provide LMIs to verify stability of your closed-loop system.
4. (2p) For what range of sampling intervals h is the system stable in this scenario?

Now we consider the system is implemented with a varying sampling interval taking two possible values h_1 and h_2 but the choice of controller to be employed becomes stochastic. When the sampling interval is h_1 the controller $\bar{K}$ is applied in sample-data fashion with probability p , and otherwise the controller applied is $\bar{K}_2$; and when the interval is h_2 the control $\bar{K}_2$ is applied in sample-data fashion with probability q , and otherwise the zero input is applied.

Question 2: (10p)

1. (3p) Create a Markov Chain that models this behavior.
2. (3p) Write down the equations of the Markovian Jump Linear System describing the dynamics of the received samples using this network model
3. (2p) Choose a stochastic stability notion and provide LMIs to verify stability of your closed-loop system.
4. (2p) Set $p = 0.2$ and $q = 0.3$. For what range of sampling intervals h is the system stable in this scenario? Comment on the differences between the range of stabilizing sampling intervals h that you obtained now with respect to the previous question.

Next we consider the case when delays are present in the networked system, but the system is no longer switching controllers but instead employs always a controller $\hat{K}$ (to be designed). Assume that the system is affected by a **constant** small delay $\tau \in [0, h)$.

In order to avoid complex eigenvalues of the matrix A , consider **only for this exercise** these other dynamics:

$$A = \begin{bmatrix} 0.3 + a - b & 0 \\ 1 & 0.5 + c \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

where a is given by the first digit, b by the 3rd digit, and c by the last digit of your student ID number.

Question 3: (10p)

1. (2p) Design a linear continuous time controller $v(t) = -\hat{K}\xi(t)$ placing the poles of the continuous closed-loop at $-2 + j$ and $-2 - j$. Redo question 2.1 from Assignment 1 with this new controller.
2. (4p) Employ the so-called Jordan form approach (as in Lecture 4) to construct a polytopic discrete-time model over-approximating the uncertain exact discrete-time closed-loop NCS dynamics.
3. (2p) Perform stability analyses, solving the relevant LMIs resulting from the previous question polytopic over-approximation. Employing this analysis, produce a plot illustrating combinations of (h, τ) as in Question 2 of Assignment 1.
4. (2p) Compare the resulting plot with the one from Q3.1 and discuss any possible differences and the possible causes for such differences.

Finally we go back to the case with no delays to design an event-triggered controller.

Question 4: (4p)

1. (1p) Consider your system being controlled by the first static controller you designed for your continuous-time system $\bar{K}$. Design a quadratic event-triggered condition

$$\phi(\xi(t), \xi(s_k)) \leq 0$$

for the system to guarantee global exponential stability of the closed-loop.

2. (2p) Simulate the resulting closed loop for various values of the σ parameter controlling the guaranteed performance of the closed-loop. Simulating for multiple initial conditions for each value of σ , compare the average amount of communications/sampling produced by each of the systems (different σ values) on a predefined fixed time-length of the simulations.

3. (1p) Let h_{avg}^* be the average inter-sample time for the value of σ in Item 2 that resulted in the lowest amount of communications/sampling. Does periodic sampling with $h = h_{\text{avg}}^*$ stabilize also your closed-loop system?

Hint: To simulate event-triggered control in continuous-time, you need to detect the triggering even while solving the ODE. The naïve way is to simulate at a very fine discrete-time rate, but this ends up being imprecise and computationally inefficient. ODE solvers in Python, Matlab, and Simulink, for example, have event-checking mechanisms that you can set up to, e.g., stop simulation at an state-dependent event. You can use this to stop simulation, update the control action, and restart the simulation in a loop. Here are some pointers for event detection:

- **Matlab:** any ODE solver, e.g., `ode45`, has an optional `options` input; look up `odeset` for implementation details (e.g., type `doc odeset` in the Matlab prompt and look for the “events” Section.)
- **Python:** similar to Matlab, `scipy` offers event-handling through an optional `events` argument. See the documentation for implementation details.
- **Simulink:** The Triggered subsystem is the recommended block to implement an event-based update of a signal.

If you are familiar with hybrid systems, particularly the jump-flow formalism, you may also consider the Matlab/Simulink toolbox HyEQ from USCB.